

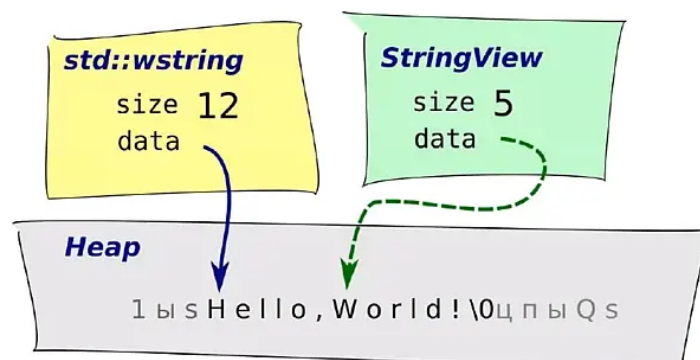
Работа со строками

StringView / BinaryStringView

StringView / BinaryStringView — это классы, которые могут ссылаться на непрерывную последовательность символов/байт.

Они подобны строкам (`std::wstring` / `std::string`), но, в отличие от последних, не хранят свою собственную копию данных. Благодаря этому у них практически бесплатные операции копирования, взятия подстроки.

StringView / BinaryStringView



На рисунке выше наглядно изображены представления `std::wstring` и `StringView` в памяти. `std::wstring` ссылается на собственный буфер, который он выделяет в куче при конструировании. При разрушении он их освобождает.

`StringView` так же ссылается на буфер с данными. Но, в отличие от `std::wstring`, он им не владеет — ссылается на чужой буфер. И, соответственно, в деструкторе `StringView` не разрушает этот буфер.

`StringView` является read-only ссылкой на строку, то есть он применим только для чтения данных строки.

Когда полезен StringView

Так как данные, на которые ссылается `StringView`, невозможно модифицировать, область использования его ограничена задачами, где выполняется только чтение строки. В реальной жизни это ограничение не делает `StringView` малоприменимым, так как в большинстве сценариев строки используются только для чтения.

При использовании `StringView` появляется возможность абстрагироваться от способа хранения данных строк, сделав интерфейсы более гибкими. Рассмотрим несколько примеров

Метод, принимающий строку только для чтения

Предположим, у нас есть некоторый метод, который принимает в параметре строку, которую он не будет модифицировать:

```
1 class DbStatement
2 {
3     public:
4         void Exec ( ??? query )
5         {
6             internal_c_driver_exec( query_data, query_size );
7         }
8     private:
9         String mName;
10 }
```

Перед программистом встает сложный вопрос: какой тип данных использовать?

1. Если использовать `const String&`, то получим паразитное копирование, если потребуется передать константную строку:

```
1 stmt.Exec( L"SELECT * FROM table" ); // паразитное выделение памяти из кучи и копирование
```

2. Если использовать `const wchar_t*`, то теряем информацию о длине и будем вынуждены вычислить `query_size` через `strlen` в runtime. Более того, этот способ неудобен при реализации query (придется иметь дело с СИ-строкой вместо удобного STL-класса).

В таком случае нам логично абстрагироваться от способа представления входной строки, воспользовавшись `StringView`:

```
class DbStatement
{
public:
    void Exec( const StringView& query )
    {
        internal_c_driver_exec( query.data(), query.size() );
    }
};

const String& GetQuery();

1. stmt.Exec( GetQuery() ); // Ок.
2. stmt.Exec( L"select 1"_sv ); // Ок.
```

В данном примере используется суффикс `_sv`, о нем будет рассказано [далее](#).

Глобальная строковая константа

Также зачастую программисту требуется объявить строковую константу. Рассмотрим способы, как он это может сделать:

1. Использовать `String`

```
1 static const String DOCUMENTS_TABLE = L"Документы";
```

Плюсы:

- удобно пользоваться константой (удобный интерфейс);
- известна длина.

Минусы:

- копируется при старте на кучу — замедляется старт и потребляется лишняя память;
- возможно неопределенное поведение — если использовать эту константу в конструкторе или деструкторе другого глобального объекта.

2. Использовать `const wchar_t*`

```
1 static const wchar_t* DOCUMENTS_TABLE = L"Документы";
```

Плюсы:

- На этапе компиляции укладывается в секцию константных данных, которая просто отображается на память при запуске и разделяется между процессами.

Минусы:

- Не известна длина, придется вычислять в runtime через `strlen`;
- Неудобно пользоваться, так как это не класс (придется использовать libc-функции `strchr`, `strdup` и т.п.).

Если используем `StringView`

```
1 static const StringView DOCUMENTS_TABLE = L"Документы"_sv;
```

Преимущества:

- никакого лишнего кода в runtime,
- хранится в RO-секции константных данных,
- удобный интерфейс.

Ограничения StringView

1. Не позволяет модифицировать данные, поскольку StringView, по своей природе, ссылка read-only.
2. Содержит НЕнультерминированную строку, вследствие этого:

- нет метода `c_str`
- нельзя использовать результат `data()` как нультерминированную строку

Если все же очень надо получить нультерминированную строку — придется копировать, причем лучше в [StackString](#).

3. Нельзя хранить StringView, ссылающийся на данные, которые могут "умереть" раньше его. Если необходимо долгосрочно запомнить данные, переданные в метод в виде StringView, необходимо их копировать в какой-то контейнер (StackString, String).

Интерфейс StringView

StringView практически полностью повторяет интерфейс `std::string`. Отличия заключаются в особенностях конструирования, а так же в StringView отсутствуют все модифицирующие операции.

Конструирование StringView

1. Можно создать из String-подобного контейнера:

```
1 | StringView( some_vector_of_wchart )
```

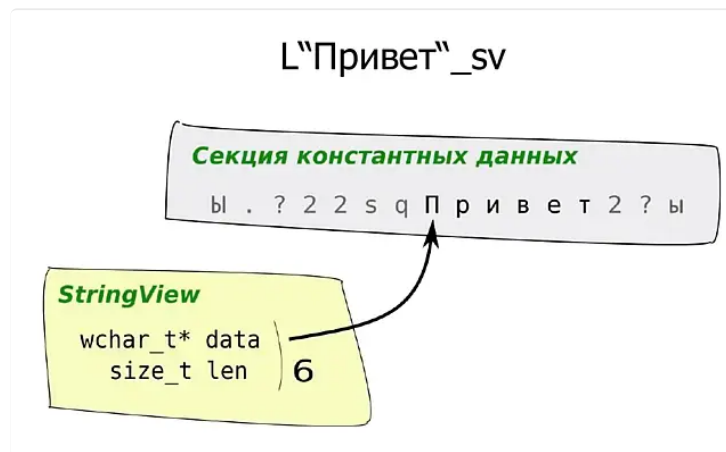
StringView может построиться из любого STL-подобного контейнера, который хранит данные в виде непрерывного массива.

Требования к такому контейнеру:

1. Должен определять следующие типы:
 - *value_type* — тип элементов, которые хранит контейнер; для контейнера, содержащего "широкие" строки (совместимого с StringView) это должно быть *wchar_t*, для контейнера с "узкой" строкой (совместимого с BinaryStringView) — *char*;
 - *size_type* — тип, используемый для представления размера данных; обычно это *size_t*;
 - *const_iterator* — тип константного итератора;
 - *iterator* — тип итератора для перебора контейнера (может совпадать с *const_iterator* для немодифицируемых контейнеров).
2. Должен содержать следующие методы
 - *data()* — возвращает указатель на свои данные (возвращает массив типа *value_type**);
 - *size()* — возвращает размер данных в контейнере (тип *size_type*);
 - *begin()* — возвращает итератор или константный итератор на начало данных (тип *iterator* или *const_iterator*);
 - *end()* — возвращает итератор или константный итератор на конец своих данных (тип *iterator* или *const_iterator*).

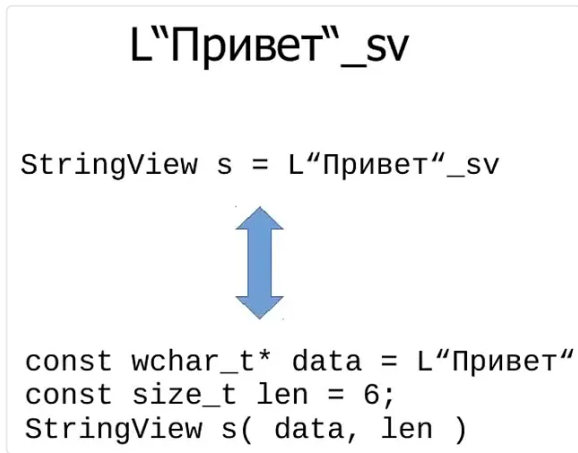
Среди стандартных контейнеров это, например, `std::string`, `std::vector`, `std::array`.

2. Создание на этапе компиляции `L"Некоторая_строка"_sv`



Компилятор создает StringView, который ссылается на указанную нами строку, расположенную в секции константных данных. Также компилятор рассчитывает длину и прикладывает этот объект. Это делается на этапе компиляции в runtime.

Таким образом, компилятор генерирует код, подобный тому, как если описать вручную константы:



3. Конструирование из gvalue-объектов запрещено:

```
1 String SomeFunc();
2 StringView sv( SomeFunc() ); // ERROR
```

Прочие методы StringView

Интерфейс подобен std::string, есть почти все методы: size, data, find и т. п. Например,

```
1 size_t pos = sv.find( L"x" );
```

Операции создания подстроки практически бесплатные (копируется указатель и целое число):

```
1 StringView substr = sv.substr( 5 );
```

Есть операция подрезания пробельных символов в начале и конце строки:

```
1 substr = substr.trimmed(); // возвращает новый StringView
```

Эта операция также является очень дешевой.

StackString / BinaryStackString

При разработке приложений очень часто возникает задача, когда нужно "склеить" строку из нескольких компонент и сдать ее для чтения в некоторую функцию. Рассмотрим самые популярные варианты её решения.

1. Конкатенация строк типа std::string

```
1 std::wstring x = ToString( some_int ) + some_string + ToString(
  another_int );
2 SomeReadFunction( StringView( x ) ); // функция только читает нашу
   строку
```

Что тут происходит?

- ToString(some_int) создает временный объект std::wstring (+1 аллокация)
- ToString(some_int) + some_string выполняет конкатенацию временной строки из первого шага с some_string. Вполне вероятно, что для этой операции не хватит места в буфере строки и придется сделать реаллокацию памяти (+1 аллокация, +1 деаллокация)
- ToString(another_int) создает еще один временный объект std::wstring (+1 аллокация)
- <tmp1> + <tmp2> конкатенирует строки, расширяя первый временный объект. Возможно, снова потребуются реаллокация буфера. После выполнения этой операции второй временный объект будет разрушен (+1 аллокация, +2 деаллокации)
- Временный объект перемещается в переменную x
- После вызова функции LogMsg наш объект x выходит из области видимости и тоже разрушается (+1 деаллокация)

Итого, мы получили четыре аллокации памяти и четыре деаллокации.

Плюсы:

- Красивый, лаконичный код
- Отработает на всех возможных входных данных

Минусы:

Очень много обращений к куче => медленно работает + фрагментирует кучу

2. Конкатенация строк на стековом буфере с использованием функций libc

```
1 wchar_t buf[ 512 ];
2 swprintf( buf, 512, L"%d%ls%d", some_int, some_string.c_str(),
  another_int );
3 SomeReadFunction( StringView( buf ) );
```

В данном случае нет обращений к куче, все происходит на стековом буфере. Но это решение имеет существенный недостаток: если строка some_string придет слишком большого размера, и результирующая строка не влезет в буфер, то swprintf обрежет результат. В итоге, в дальнейший код пойдет "битая" строка, что может привести к проблемам.

Плюсы:

Быстро работает

Минусы:

- В случае нехватки места в буфере выдаст обрезанную строку
 - Опасный интерфейс — в случае ошибки в строке формата получаем неопределенное поведение
- Таким образом, в подобных задачах необходим класс, который работает как стековый буфер, но реаллоцируется в куче в случае нехватки места.

В Wasaby Framework данный класс называется StackString - String-подобная строка со стековым буфером:

- Имеет интерфейс, практически идентичный std::string;
- Работает без аллокаций, пока данные умещаются в стековый буфер;
- Переходит на кучу, если данные не вмещаются в стековый буфер.

Используя этот класс совместно с [StringBuilder](#), получаем:

```
1 StackString< 512 > buf;
2 MakeStringBuilder( buf ) << some_int << some_string << another_int;
3 SomeReadFunction( StringView( buf ) );
```

ИТОГО: нет аллокаций, красивый код, не ломается на длинных строках.

StackString является string-подобным контейнером => его можно использовать во всех методах, которые принимают StringView, например:

```
1 | SqlQuery( StringView( query ) );
```

Форматирование строк

Одна из задач обработки строк — форматирование. В данном классе задач необходимо сформировать строку по некоторым входным данным. К ним относятся:

- Преобразование различных строк в строковое представление;
- Конкатенация строк;
- Форматирование строк по шаблону.

В данном разделе рассмотрим платформенный функционал, который позволяет решить данный класс задач.

ToString

Метод, возвращающий строковое представление переменной.

- Умеет работать с узкими и широкими строками.
- Не только возвращает новый объект, но и умеет дописывать результат в любой существующий контейнер.

Примеры использования:

1. В данном случае ToString возвращает новый объект типа String, в котором содержится строковое представление переменной count:

```
1 | String query = L"SELECT FROM" + table + L" LIMIT " + ToString( count )
```

2. В данном случае функция ToString дописывает строковое представление count в конец строки str. В таком варианте не создается новых временных объектов.

```
1 | String str = FillSomePrefix();  
2 | ToString( str, count );
```

StringBuilder

Класс, позволяющий удобно конкатенировать строки, на лету преобразуя нестроковые типы в их строковое представление.

Данный класс в некоторой степени похож на std::stringstream:

- Заливка через оператор "<<".
- Сам преобразует типы в строки, используя ToString.

Но StringBuilder имеет ряд преимуществ над std::stringstream:

- Не использует локали => быстрее преобразует типы к их строковым представлениям.
- Не аллоцирует память под внутренние нужды => быстрее работает и меньше фрагментирует кучу.
- Пишет сразу в String или переданный пользователем контейнер (std::stringstream хранит данные в своем собственном представлении и по запросу пользователя копирует их в новый объект) => существенно быстрее работает и расходует меньше оперативной памяти.
- Может работать с любым "плоским" контейнером, в котором данные хранятся в виде непрерывной последовательности байт (std::string, std::wstring, std::vector<char>, std::vector<wchar_t>, StackString, BinaryStackString и т.п).

Пример использования:

```
1 | WStringBuilder query( 512 );  
2 | query << L"SELECT FROM"_sv << table << L" LIMIT "_sv << count;  
3 |  
4 | // можно получить ссылку на строку используя метод Get
```

```

5 | SqlQuery( query.Get() );
6 |
7 | // или вообще отобразить у него строку БЕЗ копирования методом Release
8 | String s = query.Release();

```

Пример использования StringBuilder совместно с BinaryStackString:

```

1 | BinaryStackString<256> redis_command;
2 | MakeStringBuilder( redis_command ) << "GET"_sv <<
   | Encoded<Encoding::UTF8>( key );

```

Пример использования совместно с std::vector:

```

1 | std::vector<wchar_t> data;
2 | auto sb = MakeStringBuilder( data );
3 | for( int i = 10; i < 15; ++i )
4 |     sb << i;
5 | // в data будет '1','0','1','1','1','2','1','3','1','4'

```

Format

Форматирование строк по шаблону:

```

1 | boost::wformat( L"Документ %1% заблокирован" ) % doc_name

```

- Необходимо для локализации приложения.
- Более удобный для чтения код.

Недостатки boost::format

- Header-only библиотека:
 - раздувает размер бинарников;
 - существенно замедляет сборку.
- Работает через потоковые операторы <<.

Преимущества sbis::Format

- Не раздувает размер бинарников.
- Работает через ToString.
- Сам предсказывает и резервирует размер выходного буфера.
- Умеет дописывать в конец String-подобного контейнера.
- Быстро работает.

Ограничения sbis::Format

- Поддерживает только один формат плейсхолдеров:

```

1 | Format( L"Вот %1% такой"_sv, param )

```

- Строка формата может быть StringView или rvalue String-подобный контейнер.

```

1 | String query = Format( L"SELECT FROM %1% LIMIT %2%"_sv, table, count
   | );

```

ИТОГО: + 1 аллокация + красивый код

```

1 | StackString<512> query;
2 | Format( query, L"SELECT FROM %1% LIMIT %2%"_sv, table, count );

```

ИТОГО: + без аллокаций + красивый код

FormatFromRecord [deprecated]



После реализации [FormatByContext](#) функция **FormatFromRecord** считается устаревшей.

Функция форматирования строки по шаблону, берущая подстановки из Record'a и возвращающая результат в виде строки. Функция принимает только форматную строку с подстановками, указывающими на имена полей Record'a и сам Record, из которого будут взяты данные.

Места для подстановки форматной строки задаются в виде %имя_поля%. Если требуется подставить поле из вложенного Record'a, то можно написать вложенную подстановку %имя_поля_с_рекордом.имя_вложенного_поля%.

Если есть неоднозначности в именах подстановок (например, в вашем Record'e есть поле с именем Поле1.Поле2 и Поле1, являющееся вложенным Record'ом, у которого есть своё поле с именем Поле2), алгоритм устанавливает приоритетом наибольшую глубину вложения.

Как и в функции [Format](#), символ '%' задается в виде '%%', а аргументы преобразуются в строковый вид при помощи функции [ToString](#), поэтому для поддержки вашего произвольного типа нужно реализовать специализацию шаблона Stringify (предпочтительный вариант) или реализовать метод ToString.

Пример:

```
1 // Подставляем значение из поля первого уровня
2 auto format = L"Field value is %field%. Good job."_sv;
3 Record rec;
4 rec.AddInt32( L"field"_sv, 42 );
5 auto result = FormatFromRecord( format, rec );
6 // Результатом будет L"Field value is 42. Good job." );
7
8 // Подставляем вложенное значение
9 auto nested_format = L"Nested field value is %field.nested%. Good
  job."_sv;
10 Record rec;
11 Record nested_rec;
12 nested_rec.AddInt64( L"nested"_sv, 42 );
13 rec.Append( L"field"_sv, nested_rec );
14 auto result = FormatFromRecord( nested_format, rec );
15 // Результатом будет L"Nested field value is 42. Good job." );
16
17 // При неоднозначности будет выбрано более глубокое поле
18 Record rec;
19 rec.AddInt64( L"field.nested"_sv, 42 );
20 Record nested_rec;
21 nested_rec.AddInt64( L"nested"_sv, 13 );
22 rec.Append( L"field"_sv, nested_rec );
23 auto result = FormatFromRecord( nested_format, rec );
24 // Результатом будет L"Nested field value is 13. Good job." ), т.к.
  rec[ L"field"_sv ][ L"nested"_sv ] приоритетнее чем rec [
  L"field.nested" ];
```

FormatByContext

Используется для вычисления [подстановок значений в текст](#) на бизнес-логике.

В случае с простым текстом на БЛ вычисляется подстановка и возвращается строка с подставленным значением.

В качестве значений подстановок могут использоваться только [примитивные типы](#), optional примитивных типов и vector примитивных типов. Передача не предусмотренного значения или игнорируется, или бросается исключение.

"Исполнитель" подстановки на БЛ — это семейство функций sbis::FormatByContext с сигнатурами


```
enum class MissingPolicy
{
    ignore, // игнорируем отсутствие значений подстановок
    exception // бросаем исключение о не хватке значений
}
String FormatByContext( StringView text, Context const& ct, MissingPolicy
policy );
//не бросает исключений, выдает непорешенные имена переменных
pair<String, set<String>> FormatByContext( StringView text, Context const&
ct) noexcept;
```

расположенные в отдельной библиотеке, вместе с объявлением класса Context. Алгоритм работы следующий:

1. Извлекаем из переданной строки все требуемые подстановки. Используем "жадный" алгоритм, ищем минимальную подстроку находящуюся между "{" и "}". Нечитаемые символы не игнорируются и их использование в начале или конце выражения из {} считается ошибкой.
2. Выполняем вызов DataObject.Execute(переменные, context), в ответ получаем пары имя_переменной: значение.
3. Выполняем подстановки. Если в ответе метода DataObject.Execute не найдено значение, **то подстановка удаляется и не занимает места(не показываем технические артефакты пользователю, как это сейчас происходит в фреймах)**



Важно: весь выводимый текст обязательно экранируется во избежание инъекций кода

Расширение ToString / StringBuilder / Format

Для расширения необходимо:

1. Реализовать специализацию Stringify
2. Реализовать метод ToString для нужного типа строк.

Пример 1:

```
1  template<>
2  class Stringify<MyType>
3  {
4  public:
5      // для "широких" строк
6      void ToString( const Allocator<wchar_t>& alloc, const MyType& value
7  );
8      // для "узких" строк
9      void ToString( const Allocator<char>& alloc, const MyType& value );
10 }
```

здесь Allocator — абстракция, которая позволяет "оторвать" реализацию кода стрингификации от физического представления хранилища данных, куда мы пишем; это сущность, у которой мы можем запросить некоторое количество байт — перераспределиться под нужный размер.

Пример 2:

```
1  void Stringify<MyType>::ToString(
2      const Allocator<wchar_t>& alloc, const MyType& v )
3  {
4      MakeStringBuilder( alloc )
5          << v.key
6          << L": ";
7          << v.value;
8  }
```

Модификаторы ToString / StringBuilder / Format

В платформе есть модификаторы форматирования. Данные вспомогательные классы влияют на вид строкового представления типа.

В настоящее время существуют следующие платформенные модификаторы:

- Decoded/Encoded — декодирует / кодирует строку, записывая результат сразу в результирующую строку.

Пример 1:

```
1 catch( const std::exception& ex )
2 {
3     LogMsg(
4         Format( L"Поймано исключение с текстом: %1%"_sv,
5             Decoded<ExceptionDescriptionEncoding>(
6                 ex.what(),
7                 ReplaceBadSymbolsPolicy<wchar_t>());
8     ));
9 }
```

Пример 2:

```
1 const String path = GetURLPath();
2
3 std::string data;
4 auto sb = MakeStringBuilder( data );
5
6 sb << "GET "_sv << Encoded<Encoding::ASCII>( path ) << "HTTP/1.1";
7
8 http_client.send( data );
```

Парсинг строк

Второй класс задач обработки строк — это разбор строк, то есть создание типов по их строковому представлению. В данном разделе будут рассмотрены платформенные функции, помогающие решить данные задачи.

FromString

FromString преобразует строку в указанный тип

- В случае ошибок кидает DeStringifyError,
- Умеет работать с "узкими" и "широкими" строками,
- Принимает const wchar_t*, StringView и даже любой "плоский" STL-контейнер.

Поддержка StringView позволяет использовать FromString для обработки подстроки (при этом не копируя данные). Благодаря этому, FromString удобно использовать при реализации сложных парсеров.

Например: разбор строки вида "ключ-значение":

```
1 // "Key: 123"
2 int parse( const StringView& sv )
3 {
4     size_t pos = sv.find( L":" );
5     StringView val = sv.substr( pos );
6     return FromString<int>( val.trimmed() );
7 }
8
9 std::cout << parse( StringView( some_storage ) );
```

Плюс: неважно, где хранятся данные, переданные для парсинга.

Модификаторы FromString

FromString поддерживает модификаторы форматирования. Данные вспомогательные классы позволяют повлиять на поведение парсера.

В настоящее время в платформе реализован модификатор Hex — указывает парсеру, что нужно разбирать число в шестнадцатеричном виде (строки вида 0x123, 0XaBd, ABC и т. п.).

```
1 Int 32 x = FromString<Hex<Int32>>( str ); // парсит Int, ожидая на
    входе число в 16-ричном виде
```

Расширение FromString

Список поддерживаемых в FromString типов можно расширить. Для этого необходимо:

1. Реализовать специализацию DeStringify для своего типа.
2. Реализовать метод FromString для нужного типа строк.

Пример 1:

```
1 template<>
2 class DeStringify<MyType>
3 {
4 public:
5     // для "широких" строк
6     MyType FromString( const StringView& sv );
7     // для "узких" строк
8     MyType FromString( const BinaryStringView& sv );
9 };
```

Пример 2:

```
1 template<typename T>
2 Struct MyHelper {};
3
4 template<typename ValueT>
5 class DeStringify<MyHelper<ValueT>>
6 {
7 public:
8     // для "широких" строк
9     ValueT FromString( const StringView& sv );
10    // для "узких" строк
11    ValueT FromString( const BinaryStringView& sv );
12 };
```

Использованные материалы

- Вебинар [Работа со строками на C++](#)
- Документ к вебинару в .pdf [Эффективная обработка строк на C++](#)
- Документ к вебинару в .odt [Эффективная обработка строк на C++](#)